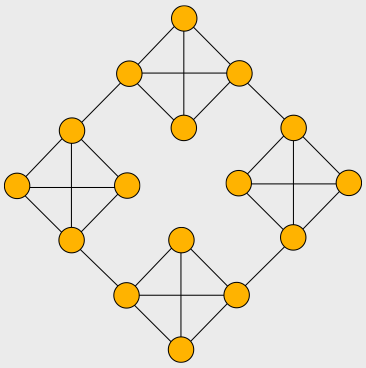
Stat 175: Homework 4

Matt Krasnow

Spring 2026

Please submit your solutions to Gradescope by **Friday, March 6th, 11:59 pm**.

Exercise 1 (Modularity and Cliques). Consider a graph G with four cliques of size 4, and exactly one edge between each neighboring clique:



Let's do Louvain community detection on this graph. Suppose that all the vertices in each separate clique have been assigned to the same community, meaning we consider four communities total.

(a) Prove that when each vertex in the same clique is assigned to the same community, then no gain in modularity can be made by moving a vertex.

(b) Draw the aggregated network.

(c) Consider the Louvain algorithm applied to the aggregated network. At each level of the algorithm, the communities of the previous level are treated as nodes of the new network. Given the initial community assignments, would you want to move any of the nodes at any level of the algorithm? Explain your reasoning.

Solution to Exercise 1

Setup. The graph G consists of four cliques C_1, C_2, C_3, C_4, each isomorphic to K_4, arranged in a ring with one bridge edge between each consecutive pair. Thus:

- Each clique has $\binom{4}{2} = 6$ internal edges.
- There are 4 bridge edges (one per adjacent clique pair in the ring).
- Total edges: $m = 4 \cdot 6 + 4 = 28$, so $2m = 56$.
- Each clique has exactly 2 bridge vertices (one bridge to each neighboring clique) and 2 non-bridge vertices.
- A bridge vertex has degree $k = 3 + 1 = 4$ (3 within its clique, 1 bridge).

- A non-bridge vertex has degree $k = 3$ (all within its clique).
- The sum of degrees within each clique is $\Sigma_C = 2 \cdot 4 + 2 \cdot 3 = 14$.

(a) **Prove that no single-vertex move improves modularity.**

Recall that the change in modularity when moving vertex v from its current community C to a different community C' is

$$\Delta Q = \frac{1}{m}\left[k_{v\to C'} - \frac{\Sigma_{C'} \cdot k_v}{2m}\right] - \frac{1}{m}\left[k_{v\to C\setminus v} - \frac{(\Sigma_C - k_v)\cdot k_v}{2m}\right],$$

where $k_{v\to X}$ denotes the number of edges from v to vertices in community X, k_v is the degree of v, and $\Sigma_X = \sum_{i\in X} k_i$ is the sum of degrees in community X.

We consider all possible vertex types and target communities.

Case 1: v is a non-bridge vertex (degree $k_v = 3$).

All three of v's neighbors lie in its own clique C. For *any* other community C':

$$k_{v\to C'} = 0, \qquad k_{v\to C\setminus v} = 3, \qquad \Sigma_{C'} = 14, \qquad \Sigma_C - k_v = 14 - 3 = 11.$$

Thus

$$\Delta Q = \frac{1}{28}\left[0 - \frac{14\cdot 3}{56}\right] - \frac{1}{28}\left[3 - \frac{11\cdot 3}{56}\right] = \frac{1}{28}\left[-\frac{3}{4}\right] - \frac{1}{28}\left[3 - \frac{33}{56}\right] = \frac{1}{28}\left[-\frac{3}{4} - 3 + \frac{33}{56}\right].$$

Computing the bracket:

$$-\frac{3}{4} - 3 + \frac{33}{56} = -\frac{42}{56} - \frac{168}{56} + \frac{33}{56} = -\frac{177}{56}.$$

So $\Delta Q = -\dfrac{177}{56\cdot 28} = -\dfrac{177}{1568} < 0$.

Case 2: v is a bridge vertex (degree $k_v = 4$) moved to the neighboring clique C' it bridges to.

Here v has 3 neighbors in C and 1 neighbor in C':

$$k_{v\to C'} = 1, \qquad k_{v\to C\setminus v} = 3, \qquad \Sigma_{C'} = 14, \qquad \Sigma_C - k_v = 14 - 4 = 10.$$

Thus

$$\Delta Q = \frac{1}{28}\left[1 - \frac{14\cdot 4}{56}\right] - \frac{1}{28}\left[3 - \frac{10\cdot 4}{56}\right] = \frac{1}{28}[1-1] - \frac{1}{28}\left[3 - \frac{5}{7}\right] = 0 - \frac{1}{28}\cdot\frac{16}{7} = -\frac{16}{196} = -\frac{4}{49} < 0.$$

Case 3: v is a bridge vertex (degree $k_v = 4$) moved to a non-neighboring community C''.

Here v has no neighbors in C'':

$$k_{v\to C''} = 0, \qquad k_{v\to C\setminus v} = 3, \qquad \Sigma_{C''} = 14, \qquad \Sigma_C - k_v = 10.$$

Thus

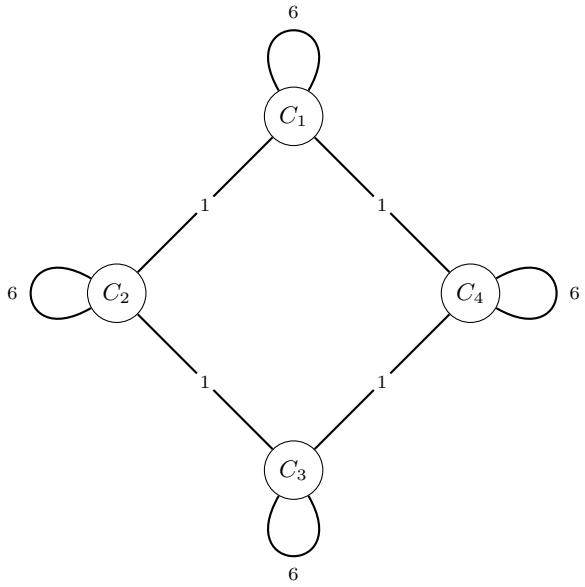
$$\Delta Q = \frac{1}{28}\left[0 - \frac{14\cdot 4}{56}\right] - \frac{1}{28}\left[3 - \frac{10\cdot 4}{56}\right] = \frac{1}{28}[-1] - \frac{1}{28}\left[\frac{16}{7}\right] = -\frac{1}{28} - \frac{16}{196} = -\frac{1}{28} - \frac{4}{49} < 0.$$

In every case, $\Delta Q < 0$. Therefore, no single-vertex move can increase the modularity. □

(b) **Draw the aggregated network.**

In the Louvain algorithm, after the first phase each community is collapsed into a single super-node. Internal edges become self-loops (with weight equal to the number of internal edges), and bridge edges between communities become weighted edges between super-nodes.

Each clique C_i becomes a node with a self-loop of weight 6 (from $\binom{4}{2} = 6$ internal edges). The four bridge edges become edges of weight 1 connecting neighboring super-nodes in a cycle.



The aggregated network is a weighted cycle C_4 on four super-nodes, where each node has a self-loop of weight 6 and each edge in the cycle has weight 1.

(c) **Would Louvain merge any communities at subsequent levels?**

At the second level, Louvain operates on the aggregated network above. Each super-node i starts as its own community. We check whether merging two adjacent super-nodes improves modularity.

In the weighted aggregated network, the *strength* (weighted degree) of each node is

$$s_i = 2 \cdot 6 + 1 + 1 = 14,$$

where the self-loop of weight 6 contributes $2 \times 6 = 12$ to the strength, and the two cycle edges each contribute 1. The total weight is $m' = \frac{1}{2}\sum_i s_i = \frac{4 \cdot 14}{2} = 28$, so $2m' = 56$.

Consider merging two adjacent super-nodes i and j (connected by an edge of weight 1). Moving node i into node j's community:

$$\Delta Q = \frac{1}{m'}\left[k_{i\to j} - \frac{s_j \cdot s_i}{2m'}\right] = \frac{1}{28}\left[1 - \frac{14 \cdot 14}{56}\right] = \frac{1}{28}\left[1 - 3.5\right] = \frac{-2.5}{28} < 0.$$

For non-adjacent super-nodes (e.g., C_1 and C_3), the edge weight between them is 0:

$$\Delta Q = \frac{1}{28}\left[0 - \frac{14 \cdot 14}{56}\right] = \frac{-3.5}{28} < 0.$$

Since every possible merge decreases modularity, **no moves are made** at the second level. The algorithm terminates, and the partition into the original four cliques is the final output.

Remark (Resolution Limit). This example illustrates the *resolution limit* of modularity. The four-clique ring is a local optimum of modularity under the Louvain algorithm: no single-vertex move (level 1) and no community merge (level 2+) improves the objective. However, one could argue that merging pairs of adjacent cliques into two larger communities might yield a partition that is globally competitive. Louvain's greedy, local nature prevents it from discovering such rearrangements, demonstrating that it can get stuck at suboptimal local maxima.

Exercise 2 (Protein Data Community Detection). In this exercise, you will explore the community structure of the Yeast protein-protein interaction network using the Louvain algorithm and spectral graph embedding.

(*a*) Download the `yeast.graphml` file from the "HW 4" folder and place it in your code file's working directory. Then run the following two commands in Python to load our dataset:

```
import igraph as ig
G = ig.Graph.Read_GraphML("yeast.graphml")
```

For the rest of this problem, we will be working with `igraph` to manipulate graphs on top of our standard `networkx`. For a manual on how to do some basic commands in this library, click the link here. Otherwise, feel free to use AI to learn new `igraph` commands. Some important ones you should use to explore the data for yourself are `G.vcount()` and `G.ecount()` (number of nodes/edges), and `G.attributes()`, `G.vs.attributes()`, and `G.es.attributes()` (attributes of the graph, nodes, and edges, respectively).

(*b*) Run the Louvain algorithm on the network and report both the modularity of the detected communities and the overall number of communities.

(*Hint: Louvain is run via* `G.community_multilevel()`*, and you can use* `dir()` *on the output to see all accessible attribute names such as modularity*)

(*c*) Visualize the aggregated network. You might need to import `cairocffi` to do this in `igraph`, or you can transfer the graph to `networkx` and use our old visualization methods.

(*d*) We want to consider another approach to community detection by embedding the nodes of the network into a low-dimensional space. To do so, we use the Laplacian Spectral Clustering method seen in lecture. Do this to obtain three different embeddings of dimensions $d = 3, 10$, and 40.

(*Hint: In Section 2, we did exactly this for only a single dimension by getting the Fiedler vector* e_2*. You can use this same method, just now with dimensions 3, 10, and 40.*)

(*e*) Now that you have your three different embeddings, run k-means clustering on them all to detect communities. Your value of k should be the number of communities you reported in part (*b*).

(*f*) Compare the community structure obtained from the Louvain algorithm with that obtained from the spectral embedding. Discuss the similarities and differences between the two.

Solution to Exercise 2

(a) We load the yeast protein-protein interaction network and restrict to the largest connected component. The resulting graph has **1458 nodes** and **1948 edges**. The graph has vertex attribute `id`; it is undirected and simple.

```
import igraph as ig
G = ig.Graph.Readchar95GraphML("yeast.graphml")
```

```
G = G.connectedchar95components().giant()
G = G.simplify()
print(G.vcount(), G.ecount())  # 1458 nodes, 1948 edges
print(G.vs.attributes())       # [ar38char39]
print(G.es.attributes())       # []
```

(b) We run the Louvain algorithm via `G.community_multilevel()`:

```
louvain = G.communitychar95multilevel()
print(louvain.modularity)  # 0.8168
print(len(louvain))         # 29 communities
```

- **Modularity**: $Q = 0.8168$.
- **Number of communities**: $k = 29$.

The high modularity indicates strong community structure. Community sizes range from 17 to 123 vertices.

(c) The aggregated network collapses each of the 29 Louvain communities into a single super-node. Edge weights between super-nodes equal the number of inter-community edges in the original graph. Figure **??** shows this aggregated network (node sizes proportional to community size).

```
agg = louvain.clusterchar95graph(combinechar95edges=dict(weight="sum"))
```

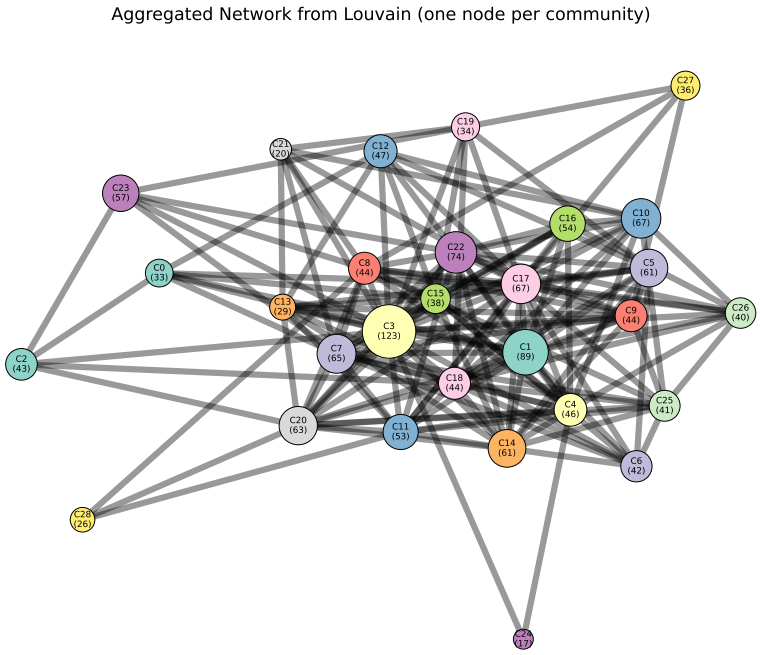


Figure 1: Aggregated network from Louvain: each node represents one of the 29 communities.

(d) We compute the unnormalized graph Laplacian $L = D - A$ and find its eigenvectors. For embedding dimension d, we use eigenvectors $e_2, e_3, \ldots, e_{d+1}$ (corresponding to the d smallest non-zero eigenvalues of L). Each vertex i is embedded as $(e_{2,i}, e_{3,i}, \ldots, e_{d+1,i}) \in \mathbb{R}^d$.

```
from scipy.sparse.linalg import eigsh
L = D - A   # graph Laplacian
eigenvalues, eigenvectors = eigsh(L, k=41, which='SM')
# For d=3: X = eigenvectors[:, 1:4]
# For d=10: X = eigenvectors[:, 1:11]
# For d=40: X = eigenvectors[:, 1:41]
```

Figure **??** plots the first two spectral coordinates $(e_{2,i}, e_{3,i})$ for each vertex, coloured by Louvain membership.

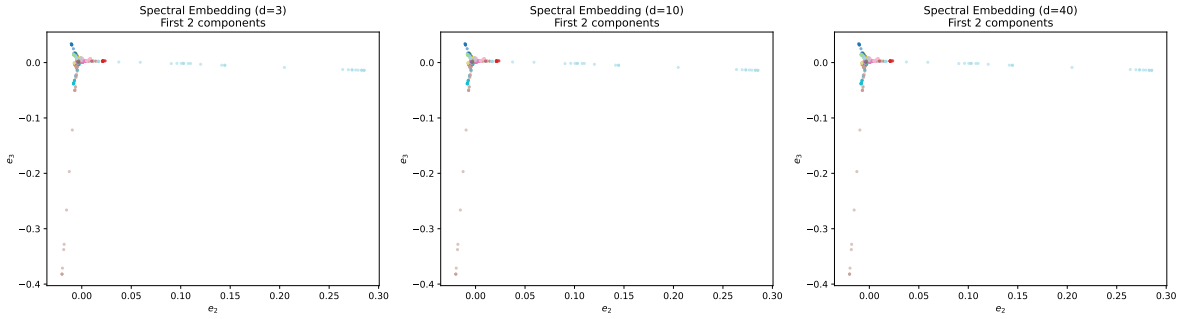


Figure 2: Spectral embeddings projected onto the first two coordinates, coloured by Louvain community.

(e) We run k-means with $k = 29$ (the number of Louvain communities) on each spectral embedding.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=29, n_init=20, random_state=42)
labels_d3  = km.fit_predict(X_3)
labels_d10 = km.fit_predict(X_10)
labels_d40 = km.fit_predict(X_40)
```

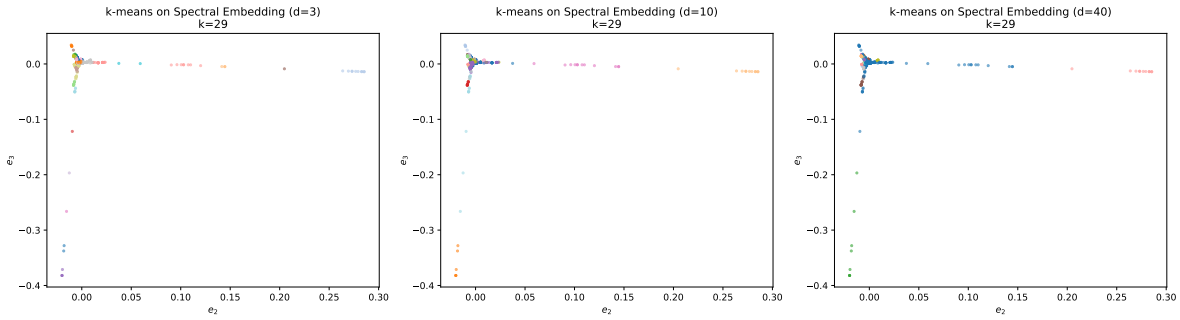


Figure 3: k-means clusters on spectral embeddings, projected onto the first two coordinates.

(f) We compare the Louvain and spectral-k-means partitions using **Normalized Mutual Information (NMI)**, **Adjusted Rand Index (ARI)**, and **modularity**.

Metric	$d = 3$	$d = 10$	$d = 40$
NMI	0.444	0.455	0.350
ARI	0.099	0.056	0.024
Modularity (spectral)	0.595	0.445	0.315
Modularity (Louvain)		0.817	

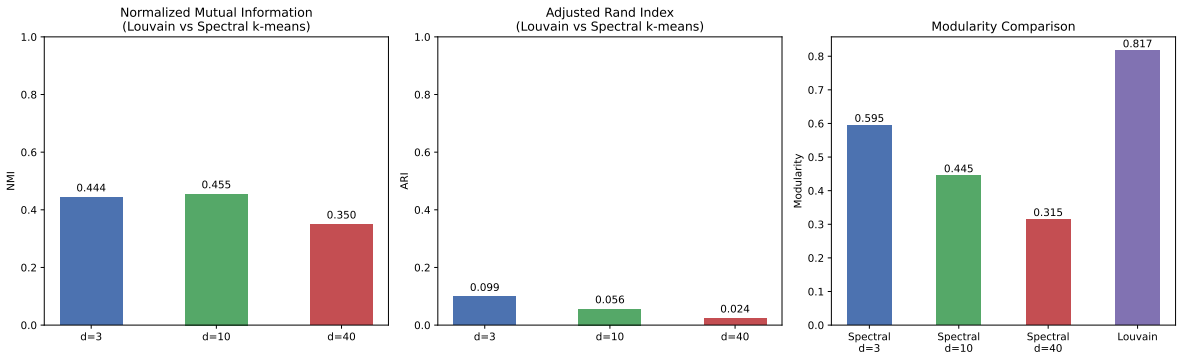


Figure 4: Quantitative comparison of Louvain vs. spectral k-means clustering.

Discussion.

- **Louvain achieves much higher modularity** ($Q = 0.817$) than any spectral-k-means partition (best $Q = 0.595$ at $d = 3$). This is expected: Louvain directly maximizes modularity, whereas spectral clustering optimizes a different objective (graph cuts via the Laplacian spectrum).
- **NMI and ARI are moderate**, indicating partial overlap between the two methods. The best agreement is at $d = 10$ (NMI ≈ 0.455), suggesting the two methods capture some of the same structure but disagree on many vertex assignments.
- **Increasing d does not always help**: $d = 10$ gives slightly higher NMI than $d = 3$, but $d = 40$ performs worst on all metrics. With 29 communities and 1458 vertices, $d = 40$ provides too many noisy dimensions for k-means (the curse of dimensionality).
- $d = 3$ **gives the best spectral modularity** (0.595), while $d = 40$ drops to 0.315. The low-dimensional embeddings capture the most informative community structure.
- Overall, Louvain is more effective for modularity-based community detection on this network, while spectral methods capture different structural features (e.g., bottleneck/cut-based communities). The two approaches are complementary.

Exercise 3 (Cut Metric). Let $G = (V, E)$ and $G' = (V, E')$ be two graphs on the same vertex set with $|V| = n$. The *cut distance* is defined as:

$$d_{\square}(G, G') = \frac{1}{n^2} \min_{\pi \in \mathbb{S}_n} \max_{S,T \subseteq V} \Big| e_G(S, T) - e_{G'}(\pi(S), \pi(T)) \Big|,$$

where $\mathbb{S}_n$ is the set of permutations of $\{1, \ldots, n\}$, and $e_G(S, T)$ denotes the number of edges that exist in G between vertex subsets S and T.

(*a*) Provide an interpretation of the cut metric. What does it capture about the global density of the graphs?

(*b*) Let (G_n) and (G'_n) be two sequences of graphs such that $d_{\square}(G_n, G'_n) \to 0$. Prove that the triangle densities converge to the same limit.

(More generally, we actually know that $|t(F, G_n) - t(F, G'_n)| \to 0$ for any motif F, where $t(F, G)$ is as defined in lecture.)

(*c*) Let (G_n) and (G'_n) be sequences of graphs sampled from graphons W and W', respectively. Assuming that you can always exchange limits and min/max, determine the limit of $d_{\square}(G_n, G'_n)$.

(*d*) Let (G_n) and (G'_n) be two sequences of sparse graphs, such that $|E_n|/n^2 \to 0$ and $|E'_n|/n^2 \to 0$. Prove that $d_\square(G_n, G'_n) \to 0$ always holds. Explain why this implies that the cut metric is uninformative for sparse graphs.

(Thus it turns out the Benjamini-Schramm metric (local convergence) is more appropriate in the sparse regime.)

(*e*) We have established that if the motif densities $t(F, G_n)$ converge for all finite graphs F, then there exists a graphon W such that $t(F, G_n) \to t(F, W)$. Furthermore, G_n converges to W in the cut metric. Explain why this identifies graphons as the natural limit objects for dense graphs.

Solution to Exercise 3

(a) **Interpretation of the cut metric.**

The cut distance $d_\square(G, G')$ measures, after optimally relabeling the vertices of one graph via a permutation $\pi \in \mathbb{S}_n$, the maximum discrepancy in the number of edges crossing any bipartite cut (S, T), normalized by n^2.

For fixed π, the quantity $|e_G(S,T) - e_{G'}(\pi(S), \pi(T))|$ measures how differently G and G' distribute edges between two vertex subsets. Maximizing over all $S, T \subseteq V$ finds the bipartite cut that best distinguishes the two graphs, and minimizing over π ensures we compare under the best possible vertex alignment.

Two graphs are close in $d_\square$ iff no bipartite cut can distinguish them after optimal vertex relabeling.

This captures *global density structure*: $d_\square(G, G') \approx 0$ means that G and G' have approximately the same edge density between every pair of large vertex subsets, up to relabeling. It is insensitive to local changes (modifying $o(n^2)$ edges does not affect the distance), but detects macroscopic differences in connectivity patterns.

(b) **Convergence of triangle densities.**

Let $t(\triangle, G) = \frac{1}{n^3} \sum_{i,j,k} a_{ij}\, a_{jk}\, a_{ik}$ denote the triangle density. Associate to each graph G_n on n vertices its step-function graphon $W_{G_n} : [0,1]^2 \to \{0,1\}$, defined by partitioning $[0,1]$ into n equal intervals $I_1, \ldots, I_n$ and setting $W_{G_n}(x,y) = a_{ij}$ for $(x,y) \in I_i \times I_j$. Then

$$t(\triangle, G_n) = t(\triangle, W_{G_n}) = \int_{[0,1]^3} W_{G_n}(x,y)\, W_{G_n}(y,z)\, W_{G_n}(x,z)\, dx\, dy\, dz.$$

The condition $d_\square(G_n, G'_n) \to 0$ implies that there exist permutations π_n such that $\|W_{G_n} - W^{\pi_n}_{G'_n}\|_\square \to 0$, where $W^{\pi_n}_{G'_n}$ is the graphon $W_{G'_n}$ with coordinates permuted according to π_n. Since relabeling does not change triangle density, $t(\triangle, G'_n) = t(\triangle, W^{\pi_n}_{G'_n})$.

Write $W = W_{G_n}$ and $U = W^{\pi_n}_{G'_n}$. By multilinearity, expanding $W = U + (W - U)$:

$$\int W(x,y)W(y,z)W(x,z) - \int U(x,y)U(y,z)U(x,z)$$

$$= \int (W-U)(x,y)\, W(y,z)\, W(x,z) + \int U(x,y)\, (W-U)(y,z)\, W(x,z) + \int U(x,y)\, U(y,z)\, (W-U)(x,z).$$

Each term has the form $\int (W-U)(s,t)\, h(s,t)\, ds\, dt$ after integrating out the third variable, where $\|h\|_\infty \le 1$. By definition of the cut norm, such integrals are bounded by $\|W - U\|_\square$. Therefore:

$$|t(\triangle, G_n) - t(\triangle, G'_n)| \le 3\, \|W_{G_n} - W^{\pi_n}_{G'_n}\|_\square \le 3\, d_\square(G_n, G'_n) \to 0.$$

Hence $t(\triangle, G_n)$ and $t(\triangle, G'_n)$ converge to the same limit. $\square$

(c) **Limit of $d_\square(G_n, G'_n)$ for graphs sampled from graphons.**

Let $G_n \sim W$ and $G'_n \sim W'$. As $n \to \infty$, the step-function graphon W_{G_n} converges to W in cut norm (a.s.), and similarly $W_{G'_n} \to W'$. Assuming we may exchange limits and min/max, the discrete minimization over $\pi \in \mathbb{S}_n$ becomes a minimization over measure-preserving bijections $\varphi : [0,1] \to [0,1]$, and the discrete max over S, T becomes a sup over measurable $S, T \subseteq [0,1]$:

$$\boxed{\lim_{n\to\infty} d_\square(G_n, G'_n) = \delta_\square(W, W') := \inf_{\varphi \text{ m.p.}} \|W - W'^{\varphi}\|_\square,}$$

where $W'^{\varphi}(x,y) = W'(\varphi(x), \varphi(y))$ and $\|f\|_\square = \sup_{S,T} \left| \int_S \int_T f(x,y)\, dx\, dy \right|$. This is the **cut distance between graphons**.

(d) **The cut metric is uninformative for sparse graphs.**

For any permutation π (in particular $\pi = \mathrm{id}$) and any $S, T \subseteq V$:

$$|e_{G_n}(S,T) - e_{G'_n}(\pi(S), \pi(T))| \le e_{G_n}(S,T) + e_{G'_n}(\pi(S), \pi(T)) \le |E_n| + |E'_n|,$$

since $e_G(S,T) \le |E(G)|$ for any S, T. Therefore:

$$\boxed{d_\square(G_n, G'_n) \le \frac{|E_n| + |E'_n|}{n^2} \to 0.}$$

This means *any* two sparse graph sequences are at cut distance zero, regardless of their structure. A sparse random graph, a sparse expander, a path, and a collection of disjoint triangles all look identical under $d_\square$ as long as $|E|/n^2 \to 0$. The $1/n^2$ normalization is calibrated for dense graphs with $\Theta(n^2)$ edges; sparse graphs with $o(n^2)$ edges are all "crushed to zero," making the cut metric **uninformative** in the sparse regime. $\square$

(e) **Graphons as natural limit objects for dense graphs.**

The established equivalence gives two equivalent notions of convergence for dense graph sequences:

- **Motif convergence:** $t(F, G_n)$ converges for every finite graph F.
- **Cut metric convergence:** G_n converges to some graphon W in $d_\square$.

Their equivalence, combined with the existence of a graphon limit, identifies graphons as the natural limit objects for the following reasons:

1. **Completeness.** Every convergent sequence of dense graphs (in either sense) has a limit that is a graphon $W \in \mathcal{W}$, the space of symmetric measurable functions $[0,1]^2 \to [0,1]$. The space of graphons, quotiented by $W \sim W'$ iff $\delta_\square(W, W') = 0$, is a *compact* metric space under $\delta_\square$. Thus graphons form the **completion** of the space of dense graphs.
2. **Universality.** Every graphon can be approximated by a sequence of finite graphs, and every convergent graph sequence has a graphon limit. The set of limit points is exactly the set of graphons (up to measure-preserving equivalence).
3. **Analytic tractability.** Graphons encode all asymptotic combinatorial information: for any motif F, $t(F, G_n) \to t(F, W) = \int_{[0,1]^{|V(F)|}} \prod_{(i,j)\in E(F)} W(x_i, x_j) \prod_i dx_i$. The passage from finite combinatorics to integration on $[0,1]^2$ makes graphons amenable to analysis, probability, and optimization.

Graphons are to dense graphs what $\mathbb{R}$ is to $\mathbb{Q}$: the completion under a natural metric ($d_\square$).

Just as every Cauchy sequence of rationals converges to a real number, every convergent sequence of dense graphs converges to a graphon, and graphons form the smallest complete space containing all dense graphs.